

Practica 8:

Estructuras de Control y Sentencias

1. Sentencia Simple es una única instrucción. En C, C ++ o Java siempre termina con ; (punto y coma). Ej.: `x = 5; printf("Hola"); int edad; .`

Sentencia Compuesta (o bloque de código) es un conjunto de sentencias simples o compuestas que se trata como una sola gran sentencia. Para lograrlo, se encierran entre llaves { } en C, Java o sangría/indentación en Python. Ej.:

```
{    int auxiliar = x; // Sentencia simple 1
    x = y; // Sentencia simple 2
    y = auxiliar; // Sentencia simple 3
}
```

2. La asignación en C siempre responde a la estructura:
 - L-value (lado izquierdo): debe ser una región de almacenamiento de memoria que sea modificable (variable, posición de arreglo, puntero)
 - R-value (lado derecho): puede ser cualquier expresión que devuelva un valor (constante, otra variable, operación matemática o el retorno de una función)

Un *Lvalue* puede actuar como *Rvalue*, pero un *Rvalue* **nunca** puede actuar como *Lvalue*

Cuando el compilador ejecuta la asignación:

1. Calcula el resultado de la expresión del R-value hasta obtener un único valor
2. Si el tipo de dato del resultado de la derecha no coincide con el tipo de la variable de la izquierda, C intenta convertir el valor al tipo del destino (izq.)
3. Se toma el valor final y se sobrescribe el contenido de la memoria apuntado por el L-value

En C la asignación de un operador que devuelve un valor. Se conoce como asignación múltiple o encadenada: `int a, b, c; a = b = c = 10; // todas las variables almacenan el valor 10.`

Para optimizar el código, C implementa asignaciones combinadas con operadores aritméticos (`+=`, `-=`, `*=`, `/=`, etc). Por ejemplo `x += 5` es equivalente a `x = x + 5`

3. Un efecto lateral es cualquier cambio en el estado del sistema que ocurre como consecuencia de evaluar una expresión. El operador de asignación (`=`) está diseñado para producir un efecto lateral: modificar el valor de una variable en la memoria (también, los operadores de incremento y decremento).

En el lenguaje C, el orden en el que se evalúan los operadores queda a criterio del compilador. Tiene libertad de resolver primero el lado derecho o izquierdo buscando

optimizar el rendimiento del procesador. Si en una misma línea de código combinas una asignación con otra operación que lee o modifica la misma variable, caes en un comportamiento indefinido. El resultado final depende del compilador. Por ejemplo: modificar y leer la misma variable en la misma sentencia

```
int x = 5
```

```
int resultado = x + (x = 10)
```

resultado vale depende de como evalúe el compilador:

- De izquierda a derecha: evalúa primero la primera x. Luego la asignación.

La operación final es $5 + 10 = 15$

- De derecha a izquierda: primero evalúa la asignación. Cambia x a 10. La operación final es $10 + 10 = 20$

4.

- Circuito Corto (Cortocircuito): el lenguaje evalúa una expresión lógica de izquierda a derecha y se detiene en el momento en que el resultado final ya está garantizado.
 - En un AND (&&), si el primer elemento es falso, toda la expresión será falsa, así que el lenguaje no mira el resto
 - En un OR (||), si el primer elemento es verdadero, toda la expresión será verdadera, así que se detiene ahí
- Circuito Largo (Evaluación Completa): el lenguaje es obligado a evaluar todas las partes de la expresión, sin importar si el primer elemento ya definió el resultado

El escenario donde el circuito largo rompe el programa y el circuito corto lo salva es al evitar una división por cero o evitar un puntero/objeto nulo. Por ejemplo, queremos verificar si una cuenta tiene saldo disponible pero la cuenta podría no existir:

- Con circuito corto (&&): `if (cuenta != null && cuenta.tieneSaldo()) { hacer algo }`
Evalúa la primera parte, como la cuenta es nula, da Falso. Salta al final del if y no da error
- Con circuito largo (&): `if (cuenta != null & cuenta.tieneSaldo()) { hacer algo }`
el lenguaje evalúa la primera parte, lo cual es falso. Pero como es circuito largo, el operador & obliga al sistema a evaluar la segunda parte. Cuando se intenta hacer `.tieneSaldo()` de una cuenta nula, el programa se rompe.

5. La regla que define Delphi, Ada y C de cómo se asocia un else cuando hay ifs anidados se la conoce como el problema del "else colgado". Si tengo: `if cond1 if cond 2 else acción;` ¿a qué if pertenece el else?

- La regla en C y Delphi lo resuelve con la regla de la proximidad más cercana: un else siempre se asocia con el if anterior más cercano que no tenga su propio else (dentro del mismo bloque de código). Por ej.:

```

if (cond1)
    if (cond2)
        accionA();
else
    accionB(); // a quien pertenece?

```

Aunque la indentación sugiera que el else es del primer if, para el compilador le pertenece al segundo if porque es el que tiene mas cerca hacia arriba. Para cambiar esto, es obligacion agregar llaves en C o begin..end en Delphi para cerrar el if y forzar al else a conectarse con el primero

- La regla en Ada es eliminar la ambigüedad con un end if; . Te obliga a ser especifico. Por ej.:

```

if cond1 then
    if cond2 then
        accionA;
    end if; // cierre de segundo if
else
    acciónB; // del primer if
end if;

```

- Python lo soluciona con la indentación obligatoria. Un else se asocia con el if que este al mismo nivel de alineación vertical (indentación). Por ej.:

```

if cond1:
    if cond2:
        print("A")
    else:
        print("B") # perteneces a cond2

```

Si se intenta poner un else mal alineado, el interprete lanzara un error de sintaxis.

6. La construcción para expresar la selección múltiple en C, se implementa mediante el switch - case:

```

switch (expresión) {
    case valor1:
        break;
    case valor2:
        break;
    default:
}

```

expresión → debe ser entero (int, char, long o enum)

Si no se coloca el break al final de un caso, el flujo del programa seguirá ejecutando los casos de abajo de forma secuencial sin importar si la condición se cumple o no.

En Pascal, Ada y Python no trabajan de la misma manera:

- En Pascal se utiliza case - of: una vez que se ejecuta el código del caso correspondiente, la estructura termina automáticamente. No existe peligro como en C de olvidarse un break. Se puede evaluar rangos (1..10) o listas de valores (1,3,5) que en C no lo permite. Al igual que C, está limitado a tipos ordinales (enteros, caracteres, booleanos, enumerados)
- En Ada, case - is, obliga a cubrir todos los valores posibles del tipo de datos que estás evaluando. Si evalúo un entero, estoy obligado a usar when others => (igual a default) para asegurar de que no quede ningún caso en el aire, de lo contrario el código no compila. No existe fall-through (igual que en Pascal). Permite rangos (1..10) y operadores de union (1 | 3 | 5)
- En Python, match - case, a diferencia de C, Pascal o Ada, se puede evaluar cualquier tipo de datos: strings, listas, diccionarios, objetos y clases. No solo compara X == Y, si no que verifica la forma de los datos, por ejemplo, si la variable es una lista que contiene dos elementos y el primero es un string, que haga esto. El caso por defecto se expresa con "case _:"

7.

- a. Si trae problemas. A nivel estructura de bloques (begin/end) está bien, pero a nivel de semántica falla porque:
 - En Pascal provoca un error de compilación, ya que prohíbe que la variable de control de un ciclo for (i) sea modificada dentro del cuerpo del ciclo. El procedimiento A la modifica (efecto colateral).
 - En Ada directamente el código no compila ya que la variable de un for no se declara explícitamente en la sección var, la declara automáticamente local como constante, solo lectura. Mientras se está ejecutando el for, Ada "tapa" la i global. El procedimiento A intenta modificar la variable i global que nunca se inicializó porque la i del for solo existe adentro del bloque el ciclo. El código falla por intentar operar con una variable global fuera de contexto.
- b.
 - Pascal aborda el problema de seguridad prohibiendo modificar la variable
 - Ada aborda el problema mediante el ámbito y la visibilidad aislando la variable del ciclo para sea físicamente imposible acceder a ella o modificarla desde un procedimiento externo

8.

- En Ada provocaría un error de compilación porque exige obligatoriamente que case cubra todos los valores posibles. Como puntos es un entero, obliga a usar

when others => . En la sintaxis, los rangos se escriben igual que en Pascal (..) pero la asignación se hace con flecha ==>:

```
case Puntos is
  when 1 .. 5 =>
    Put("No puede continuar");
  when 10 =>
    Put ("Trabajo terminado");
  when others =>
    null; // no hacer nada
end case;
```

- En C:
 1. No soporta rangos en el switch: 1..5 o 1: 5
 2. Si no se pone break, el flujo se cae

```
switch (puntos) {      // para hacer un rango hay que "apilar" los casos
                      // aprovechando la caída por falta de case
  case 1:
  case 2:
  case 3:
  case 4:
  case 5:
    printf("No puede continuar\n");
    break;
  case 10:
    printf("Trabajo terminado\n");
    break;
  default:             // opcional en C
    break;
}
```

9. La diferencia entre return y yield es como manejan la vida de la función y la memoria del sistema al devolver un valor:

- return : cuando la función encuentra return, devuelve el valor y termina. Toda la info local de la función se destruye en memoria. Si la vuelves a llamar, empieza desde 0
- yield : cuando encuentra un yield, devuelve el valor pero congela la función en ese punto. Guarda el estado de todas sus variables y la posición de ejecución. La próxima vez, la función despierta justo donde se quedo y continua hasta el siguiente yield. El uso de yield es critico cuando se trabaja con grandes volúmenes de datos o flujos de información infinitos.

10. En JS, el método `map()` es una función nativa de los arreglos (Arrays) que se utiliza para la transformación de datos. Su función principal es recorrer un arreglo, aplicarle una función modificadora a cada uno de sus elementos y devolver un arreglo nuevo con los resultados, manteniendo intacto el arreglo original.

```
const números = [1, 2, 3];
const duplicados = numeros.map(num => num * 2);
console.log(duplicados); // [2, 4, 6]
console.log(números); // [1, 2, 3] (El original no cambió)
```

Alternativas a `map()`:

1. `for` o `for..of` → mas antigua. No devuelve nada automáticamente, se debe crear un nuevo arreglo e ir empujando los elementos uno a uno. Se usa cuando se necesita detener el ciclo antes de tiempo

```
const duplicadosFor = [];
for (let num of numeros) {
    duplicadosFor.push(num * 2);
}
```

2. `Array.prototype.forEach()` → `forEach()` no devuelve nada, solo sirve para iterar y ejecutar acciones secundarias como guardar datos en una base de datos o imprimir en consola. Se usa cuando se quiere recorrer el arreglo pero no se quiere generar un nuevo arreglo transformado a partir de el

```
numeros.forEach(num => console.log(num * 2));
```

3. `Array.from()` → crea copia del arreglo (o de un Set o NodeList) y acepta una función de mapeo como segundo argumento. Se usa cuando se necesita transformar un tipo de dato que no es un arreglo puro en un arreglo modificado

```
const duplicadosFrom = Array.from(numeros, num => num * 2); // [2, 4, 6]
```

4. `Array.prototype.reduce()` → herramienta mas poderosa y versátil de los arreglos. Sirve para reducir un arreglo a un solo valor como una suma. Se usa cuando la transformación es compleja y requiere, por ejemplo, transformar y filtrar elementos al mismo tiempo en una sola pasada

```
const duplicadosReduce = numeros.reduce((acumulador, num) => {
    acumulador.push(num * 2);
    return acumulador;
}, []);
```

11. Un espacio de nombres (namespace) es un contenedor abstracto o un sistema de mapeo que se asegura de que todos los nombres de un programa (variables, funciones, clases, objetos) sean únicos para que no entren en conflicto entre sí. Es como un sistema de archivos: un espacio de nombres como la carpeta de la computadora. No se puede tener dos archivos llamados notas.txt en la misma carpeta pero si creo una carpeta llamada Universidad y otra Trabajo, puedo tener un archivo notas.txt en cada una sin problemas.

En Python, los espacios de nombres se implementan de forma dinámica mediante diccionarios internos. Los mecanismos de implementación son:

- Módulos: cada archivo con extensión .py es un módulo y genera su propio espacio de nombres. Si tienes un archivo aritmetica.py con una función sumar(), su nombre completo para el sistema es aritmetica.sumar()
- Paquetes: son carpetas que contienen múltiples archivos .py. Agrupan módulos en jerarquías más grandes. (ej. mi_proyecto.modelos.usuarios)
- Instrucciones clave: para gestionar estos espacios, utiliza import, from.. import.. y el renombrado con as

```
# Archivo: calculos.py
import aritmetica as art
from estadistica import promediar
# evitamos conflictos si 'aritmetica' y 'estadistica' tuvieran funciones con el
# mismo nombre
resultado1 = art.sumar(5, 5)
resultado2 = promediar([10, 20])
```

En Python : La regla del alcance LEGB (Local, Enclosing, Global, Built-in)

1. Ciclo de vida dinámico: los namespaces se crean en diferentes momentos. El espacio *Built-in* (funciones como print()) se crea al arrancar Python. el Global se crea al leer el módulo. Y el Local se crea cuando una función es llamada y se destruye al terminar la función
2. Es un objeto que se puede inspeccionar los espacios de nombres en tiempo de ejecución usando locals() o globals() que devuelven el diccionario real que usa Python para buscar variables

En PHP : los namespace se diseñaron para el caos de colisión de nombres que sufrían las librerías de la época

1. Declaración formal: se debe usar la palabra clave namespace al principio del archivo
2. Utiliza la barra invertida para separar las jerarquías de nombres. Por ejemplo:
\\MiProyecto\\BaseDeDatos\\Conexión

3. Si estas dentro de un namespace y quieres usar una función nativa del lenguaje como `strlen()`, la busca dentro de tu espacio actual. Si no la encuentra, cae al espacio de nombre global (nativo). Para forzar el uso de una función nativa, se antepone la barra `\strlen()`